

NUR A lecture 7

In this lecture:

Bracketing
Golden section search
Brent's method
Using derivatives in 1D
Downhill simplex
Steepest descent
Conjugate gradient methods
Quasi-Newton methods
Basics of stochastic methods

Numerical Recipes for Astrophysics A

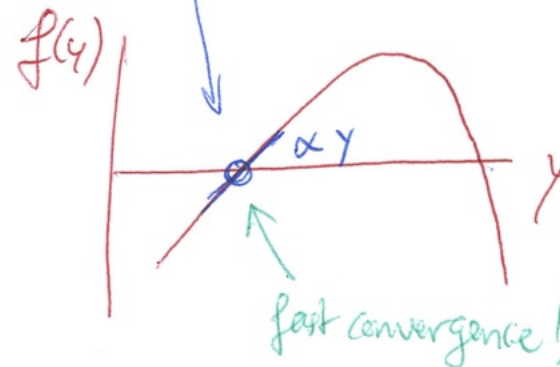
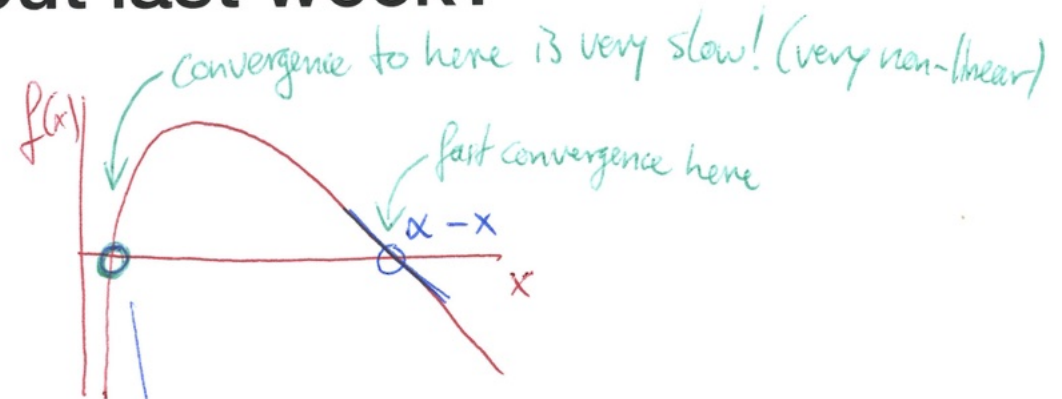
Lecture 7

Questions about last week?

$$f(x) = 5 \ln x - 10x + 20$$

$$y \equiv \ln x$$

$$f(y) = 5y - 10e^y + 20$$



What space does the function/root

"live in" locally $\Rightarrow$ search in that space

(can do each root differently, just like with splitting integrals)

Function minimization

NR Ch 10

Minimization

- Minimization=maximization=optimization

$$\max f(x) \equiv \min -f(x)$$

- Relation to root finding

$$\text{Solve } \frac{df}{dx} = 0 \quad (\text{or } \nabla f = 0)$$

← want this

- Local vs global

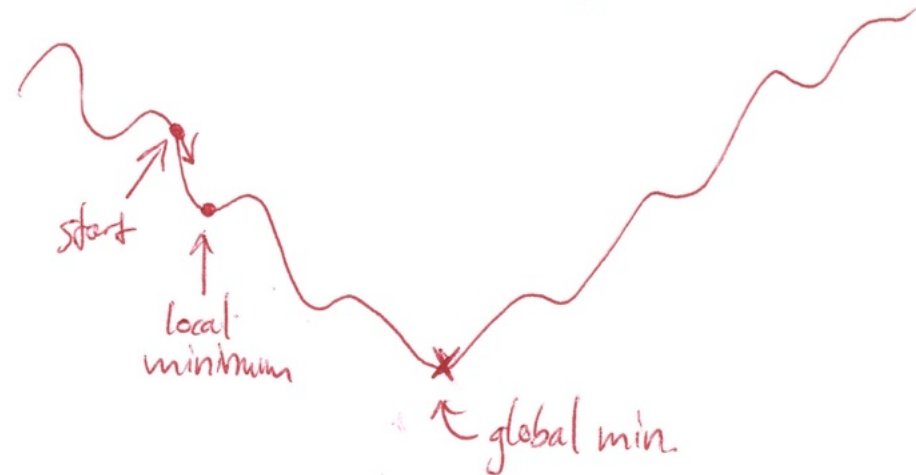
- restart often (with a different starting x)
- stochastic methods

← finding the x (or $\underline{x}$) for which the function is min. or max. (so same problem)

Sometimes works, but:

- 1) returns all extrema
- 2) need to know analytical derivs.
- 3) can be inefficient

That said, we have some uses for $\frac{df}{dx}$ and ∇f ...



Bracketing a minimum



need a 3-point bracket (or: triplet)

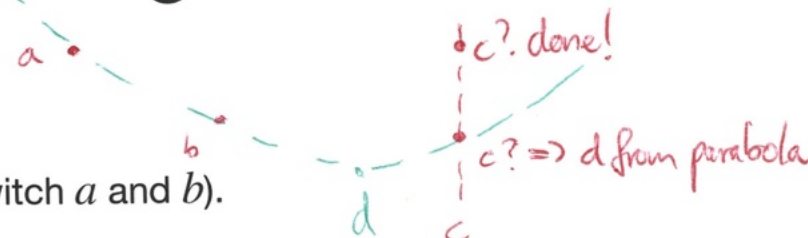
$$a < b < c, f(b) < f(a), f(b) < f(c) \Rightarrow \text{minimum in } [a, c]$$



What is the optimal place to
search for the third point
of a bracket, and how far?

Bracketing a minimum

- Finding the third point of a bracket

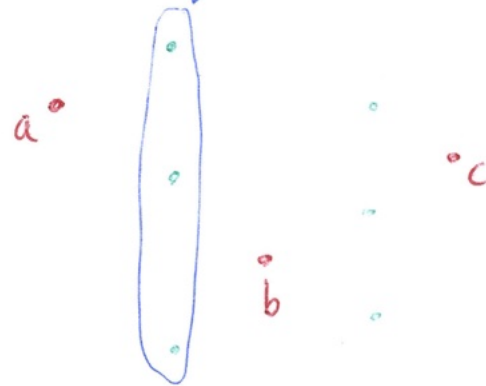


1. Ensure that $f(b) < f(a)$ (if necessary switch a and b).
2. Propose a point c at $b + (b - a)w$. A good choice for w is the golden ratio $\phi = (1 + \sqrt{5})/2 \approx 1.618$ (as we'll discuss in a bit).
3. If $f(c) > f(b)$, then we're done (bracket: $[a, b, c]$). Otherwise, use a , b and c to find a new point by fitting a parabola and setting d to the abscissa of the minimum (known analytically).
4. If d is in between b and c then we might be done: check if $f(d) < f(c)$ (bracket: $[b, d, c]$) or $f(d) > f(b)$ (bracket: $[a, b, d]$). If neither is the case, the parabola is a bad fit so just do $d = c + (c - b)w$.
5. If d is not in between b and c then we know d is beyond c ; if it's too far away (e.g. $|d - b| > 100|c - b|$), don't trust it and just take another step: $d = c + (c - b)w$.
6. If we're not done, move all points over ($a = b$, $b = c$, $c = d$) and go back to (3).

3 options: $y > f(a)$, $f(b) < y < f(a)$, or $y < f(b)$

Tightening the bracket

- Example



Taking a point in $[a, b]$ or $[b, c]$
will always result in a tighter bracket!
(Either discarding a or c)

- Accuracy limit Say min. is at $x=b$:

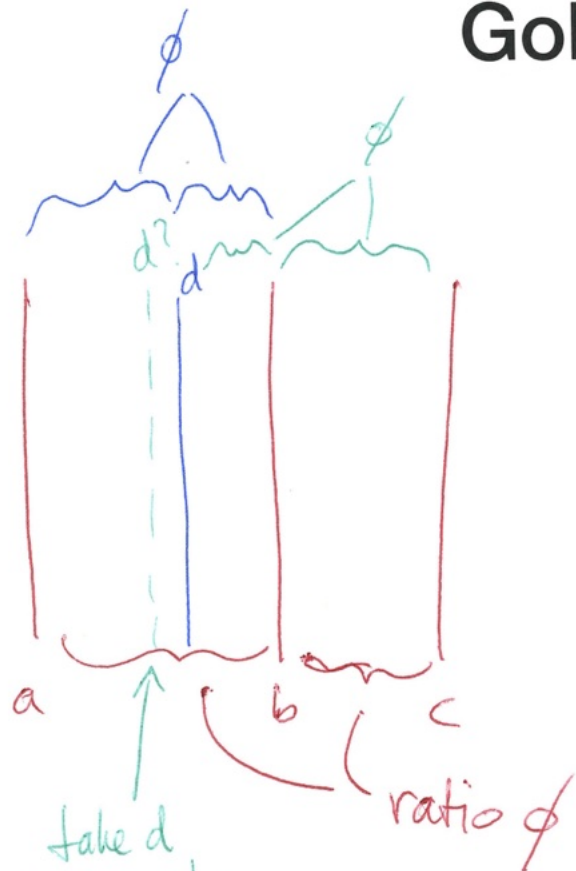
$$f(x) \approx f(b) + \frac{1}{2} f''(b)(x-b)^2 \leftarrow f'(b) = 0$$

$f(x)$ is rounded to the same value as $f(b)$ when:

$$\frac{1}{2} f''(b)(x-b)^2 < \epsilon_m |f(b)| \Rightarrow \frac{|x-b|}{|b|} < \sqrt{\frac{2|f(b)|}{b^2 f''(b)}} \sim \sqrt{\epsilon_m}$$

typical best case
↓
(for roots: $\sim \epsilon_m$)

Golden section search



take d
in the center
of $[a, b]$?

no: better worst-
case convergence
if we keep the interval ratio constant

Best worst-case behaviour

Conserve optimal interval ratio

$\Rightarrow$ self-similarity

$$\Rightarrow W = \frac{1}{1 + \phi} = 2 - \phi \approx 0.38$$

$$\text{Golden ratio } \phi = \frac{1 + \sqrt{5}}{2} \approx 1.618$$

Golden section search

Equivalent of bisection for 3-point bracket

- Iteratively tighten the bracket

1. Identify the larger interval by comparing $|c - b|$ and $|b - a|$, then choose a point d inside it in a self-similar way: $d = b + (x - b)w$ with x the other edge of the larger interval (so a or c) and $w = (1 + \phi)^{-1} = 2 - \phi \approx 0.38197$.
2. If $|c - a|$ is less than our target accuracy, we return d if $f(d) < f(b)$, or b otherwise. If $|c - a|$ does not meet the accuracy requirement, keep going.
3. If $f(d) < f(b)$, tighten towards d by setting either $a = b$ and $b = d$ (if d was between b and c), or $c = b$ and $b = d$ (if d was between a and b).
4. Otherwise, tighten towards b by setting either $c = d$ (if d was between b and c) or $a = d$ (if d was between a and b).
5. Go back to (1). For efficiency, realize that the larger interval will always be the one I didn't tighten in the previous step.

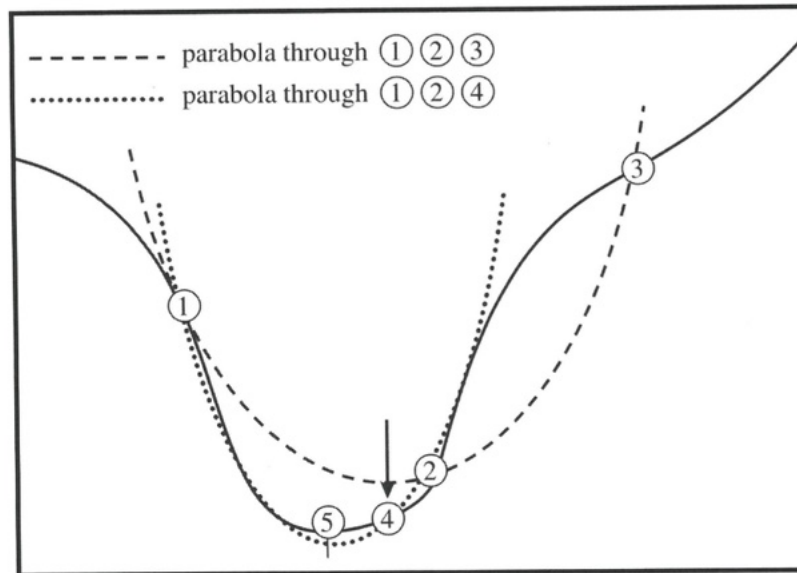
Brent's method

same Brent, similar method

- Inverse parabolic interpolation

minimum (or max.) of the parabola through a, b , and c (min. if a, b, c form a bracket!)

$$d = b - \frac{1}{2} \frac{(b-a)^2[f(b)-f(c)] - (b-c)^2[f(b)-f(a)]}{(b-a)[f(b)-f(c)] - (b-c)[f(b)-f(a)]}$$



a } outer interval
 c

b current min. (best)

w second-smallest (second-best)

v previous w

d new point

more bookkeeping due to 3-point bracket

Brent's method

1. Attempt to find a new point d through parabolic interpolation using b , w and v (initially, $w = v = b$), or a golden section step if the three points are not distinct.
2. If d is within $[a, c]$ and $|d - b|$ is smaller than the step before last, accept it (to ensure convergence while allowing a single misstep, to give the algorithm enough freedom to find its way).
3. If the step was not accepted, instead find a new point d through a golden section step.
4. Ensure that $|d - b|$ is at least $\sim \sqrt{\epsilon_m}$; if not, make it so.
 x 1.2 or so
5. Tighten the bracket: update a or c to b or d as appropriate, then update b , w and v .
 = do the bookkeeping
6. Terminate when $|c - a|$ is smaller than $2b$ times the target fractional error, or when the maximum number of iterations was reached. Otherwise, go back to (1).

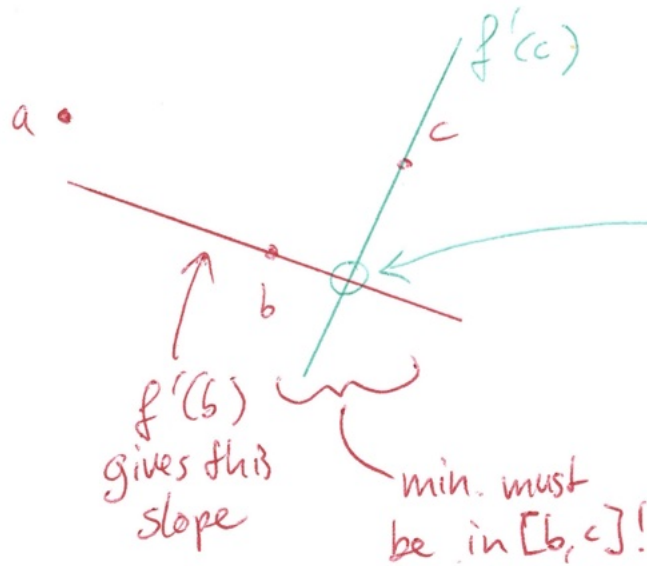
Using derivatives

- Improving our guess with $f'(x)$

E.g. $a, c, f(a), f(c), f'(a), f'(c) \Rightarrow$ cubic/polyn. fit, minimize that?
NO (dangerous)

might not even have a minimum inside $[a, c]$

- A more conservative use



$\Rightarrow$ use $f'(b)$ to know if minimum is in $[a, b]$ or $[b, c]$

$\Rightarrow$ use e.g. secant (on $f'(x)$) for your next point

If point returned is outside $[a, c]$, then use bisection (once) in either $[a, b]$ or $[b, c]$, depending on $f'(b)$.

Multidimensional: Downhill simplex

- Simplex: geometrical figure in N dim., with $N+1$ vertices

2D: 

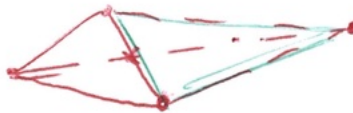
3D: 

non-degenerate = non-zero N -dim. volume!

essential!!
therefore check your
initial simplex!

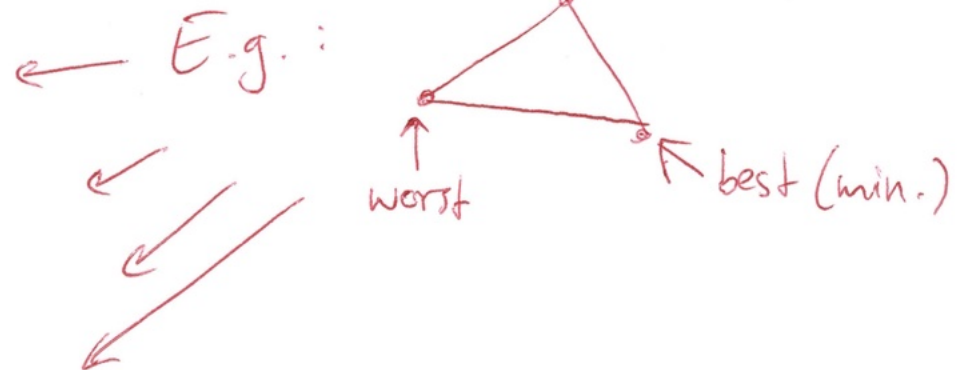
- Possible morphs:

Reflect: 

Reflect + expand: 

Contract: 

Shrink: 



Downhill simplex

most common mistake is to take the centroid of all points

$N+1$ points

1. Order the points such that $f(\mathbf{x}_0) \leq f(\mathbf{x}_1) \leq \dots \leq f(\mathbf{x}_N)$. Calculate the centroid of the first N points (so excluding the worst one):

$$\bar{\mathbf{x}} = \frac{1}{N} \sum_{i=0}^{N-1} \mathbf{x}_i.$$

2. Check if the fractional range in $f(\mathbf{x})$ (not in $\mathbf{x}!$), that is $|f(\mathbf{x}_N) - f(\mathbf{x}_0)| / [\frac{1}{2} |f(\mathbf{x}_N) + f(\mathbf{x}_0)|]$, is within target accuracy, and if so terminate ($\mathbf{x}_0$ is the best guess).

$\rightarrow \bar{\mathbf{x}} + (\bar{\mathbf{x}} - \mathbf{x}_N)$

3. Propose a new point by reflecting $\mathbf{x}_N$: $\mathbf{x}_{\text{try}} = 2\bar{\mathbf{x}} - \mathbf{x}_N$. There are now four distinct possibilities:

- I. If $f(\mathbf{x}_0) \leq f(\mathbf{x}_{\text{try}}) < f(\mathbf{x}_N)$ (new point is better but not the best), accept it (i.e. set $\mathbf{x}_N = \mathbf{x}_{\text{try}}$) and go to (4).

$\rightarrow \mathbf{x}_{\text{try}} + (\mathbf{x}_{\text{try}} - \bar{\mathbf{x}})$

- II. If $f(\mathbf{x}_{\text{try}}) < f(\mathbf{x}_0)$ (new point is the very best), propose a second point by expanding further in that direction: $\mathbf{x}_{\text{exp}} = 2\mathbf{x}_{\text{try}} - \bar{\mathbf{x}}$.
If $f(\mathbf{x}_{\text{exp}}) < f(\mathbf{x}_{\text{try}})$, accept the expanded point, otherwise accept the reflected one. Go to (4).

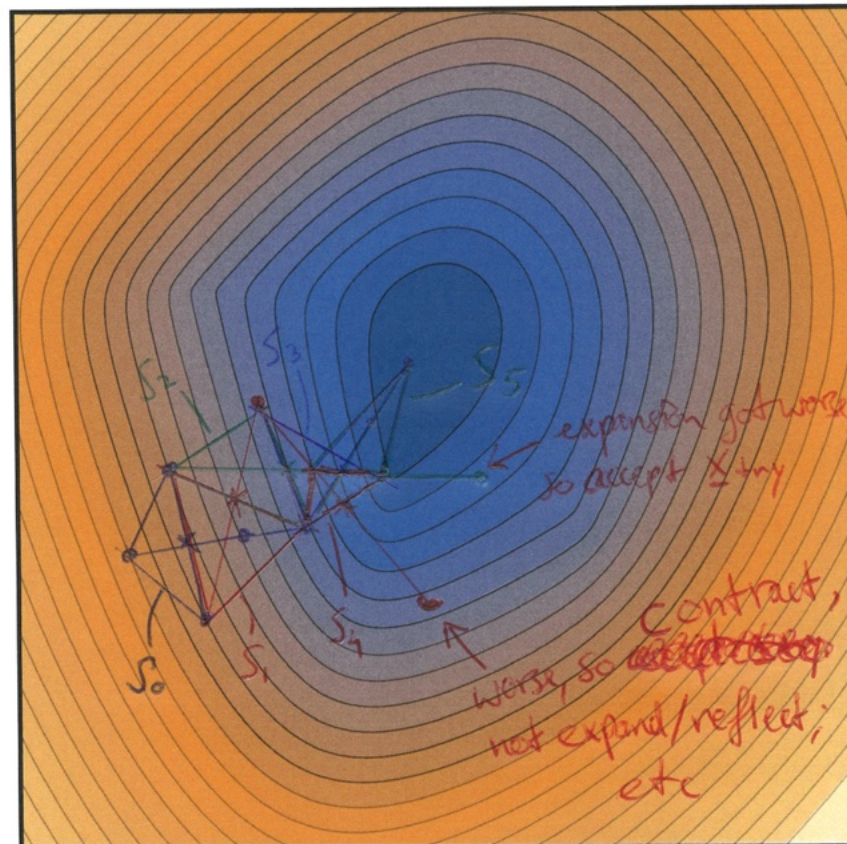
- III. Now that we know $f(\mathbf{x}_{\text{try}}) \geq f(\mathbf{x}_N)$, propose a new point by contracting instead of reflecting: $\mathbf{x}_{\text{try}} = \frac{1}{2}(\bar{\mathbf{x}} + \mathbf{x}_N)$.
If $f(\mathbf{x}_{\text{try}}) < f(\mathbf{x}_N)$, accept it and go to (4).

- IV. Since all previous options were bad, just zoom in on the best point by contracting all others to it: $\mathbf{x}_i = \frac{1}{2}(\mathbf{x}_0 + \mathbf{x}_i)$ for all $i > 0$. (probably already close

4. If you exceed some maximum number of evaluations, terminate early. Otherwise, go back to (1).

to the solution, on the scale of the current simplex)

Downhill simplex: example



Steepest descent (not great!)

- >1D: derivative $\rightarrow$ gradient

$$\underline{n} = -\underline{\nabla} f(\underline{x})$$

How far to step along $\underline{n}$? ML: step $\propto \underline{n}$, constant step $\propto$

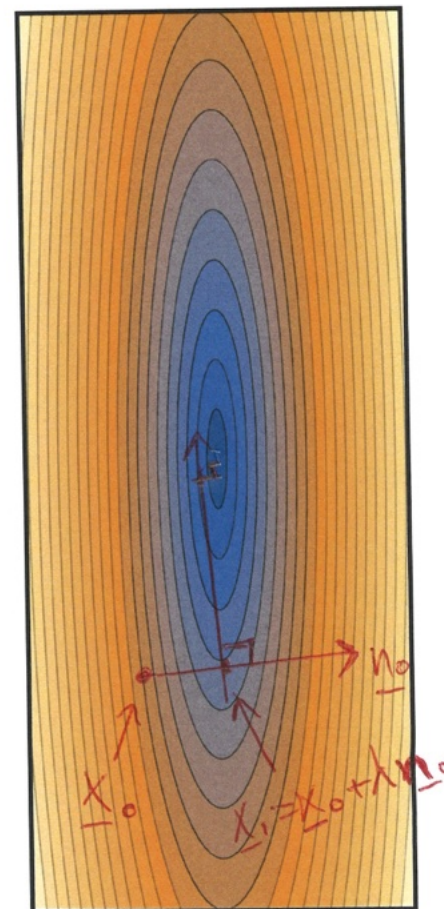
Better: minimize $f(\underline{x} + \lambda \underline{n})$ for step λ
 $\Rightarrow$ line minimization

(1D problem! Use e.g. Brent's)

- Works, but inefficient (zigzags)

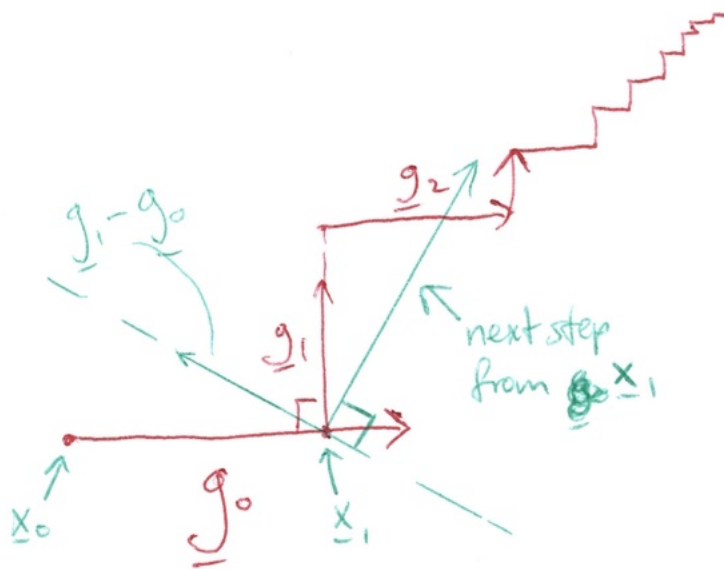
Every step is at 90° to the previous!

really bad
(fast per step, but
bad convergence overall!)



Conjugate gradient methods

- Steepest descent with shortcuts



Change in gradient:

$$\underline{g_1} - \underline{g_0} = \uparrow - \longrightarrow = \nwarrow$$

Step orthogonally to change in gradient
(i.e. don't ruin the previous progress)

Conjugate gradient methods

↙ weight for previous direction/gradients

- Polak-Ribière choice for γ_i :

1. Calculate the direction of steepest descent: $\mathbf{g}_i = -\nabla f(\mathbf{x}_i)$. Check for convergence on $\|\nabla f(\mathbf{x}_i)\|$ (or its largest value along any dimension) by comparing to its target accuracy (if it has one).

2. If $i = 0$, set $\gamma_i = 0$, else calculate γ_i as:

↙ proportional to change in gradient

$$\gamma_i = \frac{(\mathbf{g}_i - \mathbf{g}_{i-1}) \cdot \mathbf{g}_i}{\mathbf{g}_{i-1} \cdot \mathbf{g}_{i-1}}.$$

3. Set the new direction to $\mathbf{n}_i = \mathbf{g}_i + \gamma_i \mathbf{n}_{i-1}$.

↙ previous information is contained in here

4. Find the size of the step λ_i using line minimization, minimizing $f(\mathbf{x}_i + \lambda_i \mathbf{n}_i)$.

5. Set $\mathbf{x}_{i+1} = \mathbf{x}_i + \lambda_i \mathbf{n}_i$. = take the step

6. Check for convergence by comparing $|f(\mathbf{x}_{i+1}) - f(\mathbf{x}_i)| / [\frac{1}{2} |f(\mathbf{x}_{i+1}) + f(\mathbf{x}_i)|]$ to its target accuracy (if it has one).

7. Unless we converged or reached our maximum number of iterations, go back to (1).

Can I estimate the change in gradient more accurately?
Yes, if I have the second derivative.

Using the Hessian

$$B_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

dim. x dim. matrix:

- Think back to Newton's method: use second derivative

current position $\nearrow$ next step (TBD) $\nearrow$

$$f(\mathbf{x}_k + \delta \mathbf{x}) \approx f(\mathbf{x}_k) + (\nabla f(\mathbf{x}_k)) \cdot \delta \mathbf{x} + \frac{1}{2} \delta \mathbf{x} \cdot \mathbf{B} \delta \mathbf{x}$$

(Taylor x2) $\nearrow$

$$\nabla f(\mathbf{x}_k + \delta \mathbf{x}) \approx \nabla f(\mathbf{x}_k) + \mathbf{B} \delta \mathbf{x}$$

e.g. 3D: $\mathbf{B} = \begin{pmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} & \frac{\partial^2 f}{\partial x \partial z} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} & \frac{\partial^2 f}{\partial y \partial z} \\ \frac{\partial^2 f}{\partial z \partial x} & \frac{\partial^2 f}{\partial z \partial y} & \frac{\partial^2 f}{\partial z^2} \end{pmatrix}$

$$\text{Want } \nabla f(\mathbf{x}_k + \delta \mathbf{x}) = \mathbf{0} \Rightarrow \mathbf{B} \delta \mathbf{x} = -\nabla f(\mathbf{x}_k) \text{ (solve for } \delta \mathbf{x} \text{)}$$

- Step orthogonal to the change of the gradient

Issues: - B can be hard to calculate (or unknown)

- prone to divergence! (fast if I'm close, disastrous if not)

Quasi-Newton methods

$$B \underline{\delta x} = -\nabla f(\underline{x}_k)$$

- Moving in descent direction $(\nabla f(\underline{x}_k)) \cdot \underline{\delta x} < 0 \Rightarrow \underline{\delta x} \cdot (B \underline{\delta x}) > 0$

$\Rightarrow$ exactly true (for all $\underline{\delta x}$) if B is pos. definite

$\Rightarrow$ only true for B close to the min. !

- Approximate the Hessian by some H_i^{-1} such that H_i is always pos. def.

$\Rightarrow \lim_{i \rightarrow \infty} H_i = B^{-1} \Rightarrow$ guarantees we step downwards, and always converge!

Broyden-Fletcher-Goldfarb-Shanno

Quasi-Newton method: BFGS

- One good choice of update equation is BFGS:

↳ 4 people

step

$$\Delta_{i+1} \equiv \mathbf{x}_{i+1} - \mathbf{x}_i \quad \mathbf{d}_{i+1} \equiv \nabla f(\mathbf{x}_{i+1}) - \nabla f(\mathbf{x}_i)$$

$$\mathbf{u} \equiv \frac{\Delta_{i+1}}{\Delta_{i+1} \cdot \mathbf{d}_{i+1}} - \frac{\mathbf{H}_i \mathbf{d}_{i+1}}{\mathbf{d}_{i+1} \cdot \mathbf{H}_i \mathbf{d}_{i+1}}$$

the "update equation"

$$\mathbf{H}_{i+1} = \mathbf{H}_i + \frac{\Delta_{i+1} \otimes \Delta_{i+1}}{\Delta_{i+1} \cdot \mathbf{d}_{i+1}} - \frac{(\mathbf{H}_i \mathbf{d}_{i+1}) \otimes (\mathbf{H}_i \mathbf{d}_{i+1})}{\mathbf{d}_{i+1} \cdot \mathbf{H}_i \mathbf{d}_{i+1}} + (\mathbf{d}_{i+1} \cdot \mathbf{H}_i \mathbf{d}_{i+1}) \mathbf{u} \otimes \mathbf{u}$$

$\otimes$: outer product, $(\underline{v} \otimes \underline{w})_{ij} = v_i w_j$

Quasi-Newton method: algorithm

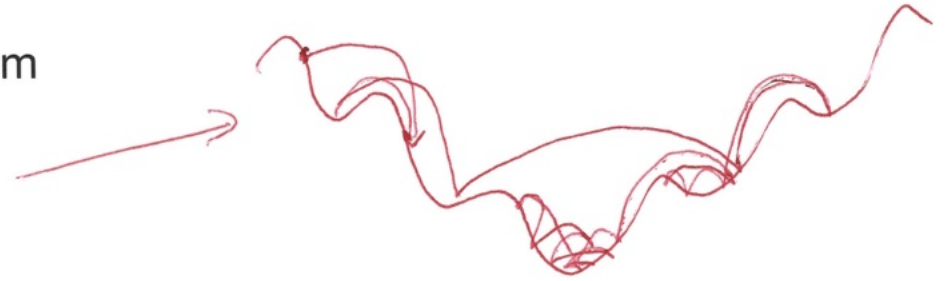
1. Get the direction of the next step: $\mathbf{n}_i = -\mathbf{H}_i \nabla f(\mathbf{x}_i)$ (for the first step, $\mathbf{H}_0 = \mathbf{I}$).
2. Full line minimization is wasteful here – instead assume $\lambda_i = 1$, check if the step decreases f significantly, otherwise lower λ_i (to at most 0.1). (Alternatively, if the $\lambda_i = 1$ result is not great do line minimization with a very generous tolerance.)
3. Set $\mathbf{x}_{i+1} = \mathbf{x}_i + \lambda_i \mathbf{n}_i$ and $\Delta_{i+1} = \lambda_i \mathbf{n}_i$. Check for convergence on the maximum (relative) step size by comparing to its target accuracy, if there is one.
4. If we're not yet converged, calculate $\mathbf{d}_{i+1}$. Check for convergence on $\|\nabla f(\mathbf{x}_{i+1})\|$ (or its largest value along any dimension) by comparing to the target accuracy, if there is one.
5. If we're not yet converged, calculate $\mathbf{u}$ and $\mathbf{H}_{i+1}$. Unless we reached our maximum number of iterations, go back to (1).

One of the best non-stochastic minimization routines that exist!

Stochastic methods

- Attempt at solving the local vs global problem

Jump around semi-randomly



- Simulated annealing (e.g. cooling metal).

not exam material | Define an "energy" difference $\Delta E_i = f(x_{i+1}) - f(x_i)$, accept x_{i+1} as the next step if $\Delta E_i < 0$ or with probability $p \sim \exp(-\Delta E_i / [k_b T])$ (just set $k_b = 1$) $\Rightarrow$ slowly reduce T with time.
E.g. $T_{i+1} = (1 - \epsilon) T_i$ with $\epsilon \sim 0.01$, or $T_i = T_0 \left(1 - \frac{i}{i_{\max}}\right)^\alpha$ with $\alpha \approx 2$.
How well this works also depend on how you pick new x_{i+1} to propose — see lecture 9
for more info on this when we discuss —

- Monte Carlo Markov Chain $\leftarrow$ in 2 weeks! statistics groundwork first

$\uparrow$ another stochastic minimization routine

**Next week:
Fitting and modelling data**